R Fundamentals

Table of contents I

1. First Encounter
2. Vectors
   - Atomic Vectors
   - Creating Vectors
   - Indexing and Selection
   - Vector Properties
3. Operators
4. Common Functions
5. Data Frames
   - Basics
   - Extracting and Filtering
   - Modifying Data Frames
   - Split–Apply–Combine
6. Lists
   - List Basics
   - List–Data Frame Connection
7. Matrices
8. Programming
   - Functions

Table of contents II

- Closures and Lexical Scoping
- Control Flow
- Loops
- Pipe Operator

9. The Apply Family
10. String Operations
11. Regular Expressions
12. Graphics
13. Date and Time
14. File I/O
15. Error Handling
16. Useful Utilities
17. Advanced Topics

First Encounter

R as a Calculator

- R as a calculator — show the numerical result of $1 + 2 \times \frac{3}{4} - 5 + 6 \sin \frac{\pi}{2} - \sqrt[8]{7}$

```r
1 + 2 * (3 / 4) - 5 + 6 * sin(pi / 2) - 7^(1/8)
```

```
# [1] 2.224627
```

- Print the content of the built-in dataset `Nile`

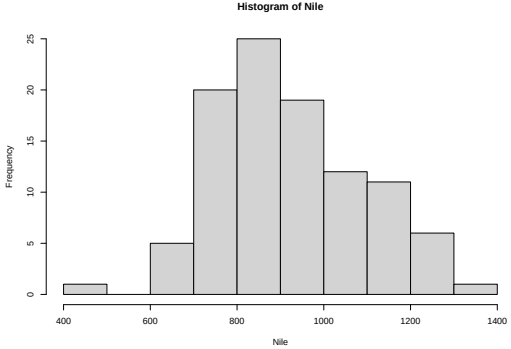
```r
Nile
```

```
# Time Series:
# Start = 1871
# End = 1970
# Frequency = 1
#   [1] 1120 1160  963 1210 1160 1160  813 1230 1370 1140  995  935 1110  994 1020
#  [16]  960 1180  799  958 1140 1100 1210 1150 1250 1260 1220 1030 1100  774  840
#  [31]  874  694  940  833  701  916  692 1020 1050  969  831  726  456  824  702
#  [46] 1120 1100  832  764  821  768  845  864  862  698  845  744  796 1040  759
#  [61]  781  865  845  944  984  897  822 1010  771  676  649  846  812  742  801
#  [76] 1040  860  874  848  890  744  749  838 1050  918  986  797  923  975  815
#  [91] 1020  906  901 1170  912  746  919  718  714  740
```

A First Glance at Plots

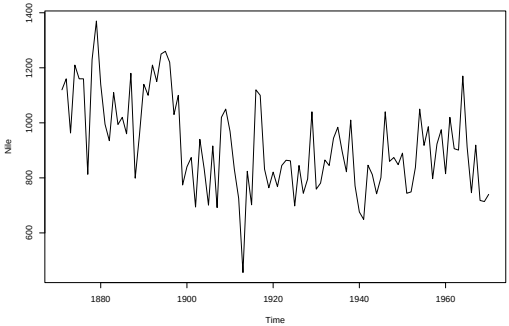
- Histogram of `Nile`

```
hist(Nile)
```



- Time series plot of `Nile`

```
plot(Nile)
```



A First Glance at Vectors

```r
Nile[2]          # the 2nd item
```

```
# [1] 1160
```

```r
Nile[2:5]        # subsequence from 2nd to 5th
```

```
# [1] 1160  963 1210 1160
```

```r
x <- c(10, 20, -1, 3)   # create a vector with c()
length(x); mean(x)
```

```
# [1] 4
# [1] 8
```

```r
x > 5                # logical comparison
```

```
# [1]  TRUE  TRUE FALSE FALSE
```

```r
sum(x > 5)           # count TRUEs
```

```
# [1] 2
```

Vectors

Vector Types and Scalars

- Flavors: **atomic** (all elements same type), **generic** (lists, elements can differ)
- Primary atomic types: `logical`, `integer`, `double`, `character`; integer + double = numeric
- Scalars are simply vectors of length 1

```r
v_logical   <- c(TRUE, FALSE)
v_integer   <- c(1L, 6L, 10L)
v_double    <- c(1, 2.5, 4.5)
v_character <- c('these', 'are', 'characters')
typeof(v_logical); typeof(v_integer); typeof(v_double); typeof(v_character)
```

```
# [1] "logical"
# [1] "integer"
# [1] "double"
# [1] "character"
```

```r
# concatenation of atomic vectors yields flat atomic vectors
(v <- c(c(1, 2), c(3, 4)))
```

```
# [1] 1 2 3 4
```

NA and NULL

- NA (Not Applicable): value **exists** but is **unknown**
- NULL: value **does not exist**

```r
NA > 5; 20 * NA; !NA              # NA is infectious!
```

```
# [1] NA
# [1] NA
# [1] NA
```

```r
NA ^ 0; NA | TRUE; NA & FALSE     # some exceptions
```

```
# [1] 1
# [1] TRUE
# [1] FALSE
```

```r
x <- c(NA, 5, NA, 10)
x == NA           # WRONG way to test (always NA)
```

```
# [1] NA NA NA NA
```

```r
is.na(x)          # CORRECT way to test
```

```
# [1]  TRUE FALSE  TRUE FALSE
```

Coercion Hierarchy

- Automatic coercion: **character** > **double** > **integer** > **logical**

```r
typeof(c('a', 1))      # character wins
```

```
# [1] "character"
```

```r
c(1, FALSE)             # logical -> double
```

```
# [1] 1 0
```

```r
c(TRUE, 1L)             # logical -> integer
```

```
# [1] 1 1
```

```r
# explicit coercion with as.*()
x <- c(TRUE, FALSE, TRUE, TRUE); as.numeric(x)
```

```
# [1] 1 0 1 1
```

```r
as.integer(c('1', '1.5', 'a'))   # 'a' -> NA with warning
```

```
# [1]  1  1 NA
```

Type Testing and Coercion Functions

Testing	Coercion
`is.logical(x)`	`as.logical(x)`
`is.integer(x)`	`as.integer(x)`
`is.double(x)`	`as.double(x)`
`is.numeric(x)`	`as.numeric(x)`
`is.character(x)`	`as.character(x)`
`is.na(x)`, `is.null(x)`	—
`is.vector(x)`	`as.vector(x)`
`is.list(x)`	`as.list(x)`
`is.data.frame(x)`	`as.data.frame(x)`
`is.matrix(x)`	`as.matrix(x)`
`is.factor(x)`	`as.factor(x)`

```r
x <- c(3, 5, 7, 8)
is.integer(x); is.numeric(x)
```

```
# [1] FALSE
# [1] TRUE
```

```r
as.integer(5.7); as.integer(-5.7)
```

```
# [1] 5
# [1] -5
```

Sequences — : and seq()

```r
5:10; 5:1
```

```
# [1]  5  6  7  8  9 10
# [1] 5 4 3 2 1
```

```r
i <- 5; 1:i-1; 1:(i-1)   # note operator precedence!
```

```
# [1] 0 1 2 3 4
# [1] 1 2 3 4
```

```r
seq(from = 11, to = 30, by = 3)
```

```
# [1] 11 14 17 20 23 26 29
```

```r
seq(from = 1.1, to = 2, length = 10)
```

```
#  [1] 1.1 1.2 1.3 1.4 1.5 1.6 1.7 1.8 1.9 2.0
```

```r
# seq_along and seq_len: safer alternatives for loops
x <- c(9, 5, 17); seq_along(x)
```

```
# [1] 1 2 3
```

```r
x <- NULL; seq_along(x)   # integer(0), not 1:0 !
```

Repetition: rep()

```r
rep(3, 5)
```

```
# [1] 3 3 3 3 3
```

```r
rep(c(10, 15, 3), 4)
```

```
#  [1] 10 15  3 10 15  3 10 15  3 10 15  3
```

```r
rep(c(10, 15, 3), each = 4)   # interleave with 'each'
```

```
#  [1] 10 10 10 10 15 15 15 15  3  3  3  3
```

```r
rep(1:4, 1:4)                 # variable repetition counts
```

```
#  [1] 1 2 2 3 3 3 4 4 4 4
```

```r
rep(c('C', 'G', 'H'), c(4, 5, 2))
```

```
#  [1] "C" "C" "C" "C" "G" "G" "G" "G" "G" "H" "H"
```

Other Generators: `sample()`, `sequence()`

```r
set.seed(42)
sample(1:6, 10, replace = TRUE)     # dice rolls
```

```
#  [1] 1 5 1 1 2 4 2 2 1 4
```

```r
sample(letters[1:20], 5)            # no replacement
```

```
# [1] "e" "n" "r" "o" "c"
```

```r
sequence(c(6, 7, 4), from = c(8, 1, 9), by = c(-1, 1, -2))
```

```
#  [1] 8 7 6 5 4 3 1 2 3 4 5 6 7 9 7 5 3
```

Numeric Indexing

```r
fib <- c(0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144)
fib[1]; fib[5]; fib[2:5]; fib[c(2, 4, 8, 1)]
```

```
# [1] 0

# [1] 3

# [1] 1 1 2 3

# [1]  1  2 13  0
```

```r
# 'minus' means 'exclude'; no mixing of positive/negative!
fib[-3]; fib[-(5:9)]
```

```
#  [1]   0   1   2   3   5   8  13  21  34  55  89 144

# [1]   0   1   1   2  34  55  89 144
```

```r
# negative indexing to build new vectors
x <- c(10, 20, -1, 3, 5, 6, 30, 9)
(y <- x[-2])                    # remove 2nd element
```

```
# [1] 10 -1  3  5  6 30  9
```

```r
(z <- x[c(-2, -3, -6)])         # remove 2nd, 3rd, 6th
```

```
# [1]
```

Logical Indexing

```r
fib[fib < 10]                          # elements < 10
```

```
# [1] 0 1 1 2 3 5 8
```

```r
fib[fib %% 2 == 0 | fib %% 3 == 0]     # multiples of 2 or 3
```

```
# [1]   0   2   3   8  21  34 144
```

```r
fib[c(TRUE, FALSE)]                     # odd-indexed (recycling!)
```

```
# [1]   0   1   3   8  21  55 144
```

```r
v <- c(2, 8, 1, 7, -11, 6, 20, 3, 12, 35, 21, 3, 5, 7, -2)
v[v > median(v)]                            # above median
```

```
# [1]  8  7 20 12 35 21  7
```

```r
v[v < quantile(v, 0.05) | v > quantile(v, 0.95)]  # extreme 5%
```

```
# [1] -11  35
```

```r
v[abs(v - mean(v)) > sd(v)]                 # beyond 1 SD
```

```
# [1] -11  20  35  21
```

which(), which.min(), which.max()

```r
which(Nile > 1200)            # positions where TRUE
```

```
# [1]  4  8  9 22 24 25 26
```

```r
Nile[Nile > 1200]             # the values themselves
```

```
# [1] 1210 1230 1370 1210 1250 1260 1220
```

```r
x <- c(5, 3, 8, 1, 9, 2)
which.min(x); which.max(x)   # index of first min/max
```

```
# [1] 4
# [1] 5
```

Removing NA from Vectors

```r
v <- c(21, 2, 3, NA, 5, NULL, NA, 32, 55)
v[!is.na(v)]                    # NULL is already dropped by c()
```

```
# [1] 21  2  3  5 32 55
```

```r
na.omit(v)                      # alternative; returns with attr
```

```
# [1] 21  2  3  5 32 55
# attr(,"na.action")
# [1] 4 6
# attr(,"class")
# [1] "omit"
```

Recycling Rule

- In operations involving two vectors of different lengths, R **repeats** the shorter one:

```r
c(2, 4, 3) + c(6, 10, -1, 3, 5)
```

```
# [1]  8 14  2  5  9
```

```r
# useful applications
x <- 1:5
x + 10            # add a constant (scalar = length-1 vector)
```

```
# [1] 11 12 13 14 15
```

```r
x * c(1, -1)      # alternate signs
```

```
# [1]  1 -2  3 -4  5
```

`all()`, `any()`, and Vector Equality

```r
x <- 1:100; any(x > 200); all(x > 0)
```

```
# [1] FALSE
# [1] TRUE
```

```r
# == is element-wise; don't use for whole-vector equality!
x <- 1:3; y0 <- c(1, 2, 3); y1 <- c(1, 3, 4)
x == y0; x == y1
```

```
# [1] TRUE TRUE TRUE
# [1]  TRUE FALSE FALSE
```

identical() and all.equal()

```r
x <- 1:3; y0 <- c(1, 2, 3)
identical(x, y0)          # FALSE: integer vs double!
```

```
# [1] FALSE
```

```r
typeof(x); typeof(y0)
```

```
# [1] "integer"
# [1] "double"
```

```r
all.equal(x, y0)          # TRUE (with tolerance)
```

```
# [1] TRUE
```

```r
all.equal(x, c(1, 3, 4)) # shows difference
```

```
# [1] "Mean relative difference: 0.4"
```

Named Vectors

```r
# 3 ways to name a vector
x <- c(a = 1, b = 2, c = 3)
x <- 1:3; names(x) <- c('a', 'b', 'c')
x <- setNames(1:3, c('a', 'b', 'c'))
```

```r
# access by name
x['b']; x[c('a', 'c')]
```

```
# b
# 2
```

```
# a c
# 1 3
```

Attributes and structure()

```r
a <- 1:3; attr(a, 'x') <- 'ab'; attr(a, 'y') <- 4:6
str(attributes(a))
```

```
# List of 2
#  $ x: chr "ab"
#  $ y: int [1:3] 4 5 6
```

```r
a <- structure(1:3, x = 'ab', y = 4:6)
str(attributes(a))
```

```
# List of 2
#  $ x: chr "ab"
#  $ y: int [1:3] 4 5 6
```

Factors

- Factors represent **categorical variables** with a fixed set of levels.

```r
(x <- factor(c('a', 'b', 'b', 'a')))
```

```
# [1] a b b a
# Levels: a b
```

```r
typeof(x); levels(x)
```

```
# [1] "integer"
# [1] "a" "b"
```

```r
# ordered factors
(size <- factor(c('S','M','L','M','S'),
    levels=c('S','M','L'), ordered=TRUE))
```

```
# [1] S M L M S
# Levels: S < M < L
```

```r
size > 'S'
```

```
# [1] FALSE  TRUE  TRUE  TRUE FALSE
```

Selecting Even/Odd Indexed Elements

```r
x <- 1:10
x[x %% 2 == 0]                    # even values
```

```
# [1]  2  4  6  8 10
```

```r
x[seq_along(x) %% 2 == 0]         # even-indexed elements
```

```
# [1]  2  4  6  8 10
```

```r
x[c(FALSE, TRUE)]                  # same, using recycling
```

```
# [1]  2  4  6  8 10
```

Appending and Inserting into Vectors

```r
v <- c(1, 2, 3); it <- c(6, 7, 8)
c(v, it)                          # concatenate
v[length(v) + 1] <- 42; v       # auto-extend (NAs fill gaps)
```

```r
append(1:10, 99, after = 5)   # insert 99 after position 5
```

```
#  [1]  1  2  3  4  5 99  6  7  8  9 10
```

```r
append(1:10, 99, after = 0)   # insert at the beginning
```

```
#  [1] 99  1  2  3  4  5  6  7  8  9 10
```

Operators

Operator Table

Operators	Definition
`+ - * /`	plus, minus, times, divide
`%/% %% ^`	integer quotient, modulo, power
`> >= < <=`	greater, greater-or-equal, less, less-or-equal
`== !=`	equals, not equals
`! & |`	not, and (elementwise), or (elementwise)
`<- ->`	assignment
`$`	list/data.frame element access
`:`	sequence creation
`~`	model formulae
`%*%`	matrix product
`%in%`	test matching
`&& ||`	short-circuit and, or (scalar only)
`::`	namespace access
`xor(a, b)`	elementwise exclusive OR
`<<-`	global / parent-env assignment
`|>`	native pipe (R 4.1+)

Operator Precedence (High to Low)

Precedence	Operators
1 (highest)	`::`, `:::`
2	`$`, `@`
3	`[`, `[[`
4	`^` (right-associative)
5	unary `-`, `+`
6	`:`
7	`%any%` (including `%*%`, `%/%`, `%%`, `%in%`)
8	`*`, `/`
9	`+`, `-`
10	`<`, `>`, `<=`, `>=`, `==`, `!=`
11	`!`
12	`&`, `&&`
13	`|`, `||`
14	`|>`
15	`~`
16 (lowest)	`<-`, `<<-`, `->`, `->>`

%in% and match()

```r
x <- c(1, 5, 3, 8, 10)
x %in% c(3, 5, 7)      # which elements of x are in the set?
```

```
# [1] FALSE  TRUE  TRUE FALSE FALSE
```

```r
# match() returns the position of first match (NA if not found)
match(c(3, 5, 99), x)
```

```
# [1]  3  2 NA
```

```r
# practical: filter data frame rows by a set of values
tg <- ToothGrowth
tg[tg$dose %in% c(0.5, 2), ][1:6, ]
```

```
#    len supp dose
# 1  4.2   VC  0.5
# 2 11.5   VC  0.5
# 3  7.3   VC  0.5
# 4  5.8   VC  0.5
# 5  6.4   VC  0.5
# 6 10.0   VC  0.5
```

Common Functions

Math Functions

Function	Definition
`abs(x)`	absolute value
`floor(x)`, `ceiling(x)`, `trunc(x)`	rounding toward $-\infty$, $+\infty$, 0
`round(x, digits=0)`	round (banker's rounding)
`signif(x, digits=6)`	significant digits
`sqrt(x)`, `exp(x)`	$\sqrt{x}$, e^x
`log(x)`, `log10(x)`, `log2(x)`	natural, base-10, base-2 log
`factorial(x)`, `choose(n, m)`	$x!$, $\binom{n}{m}$
`sin, cos, tan, asin, acos, atan`	trig functions
`beta(a, b)`, `gamma(x)`	special functions

Examples: Math Functions

```r
abs(-1); floor(5.7); ceiling(-5.7); trunc(-5.7)
```

```
# [1] 1
# [1] 5
# [1] -5
# [1] -5
```

```r
round(5.7); round(5.5); round(2.5)  # banker's rounding!
```

```
# [1] 6
# [1] 6
# [1] 2
```

```r
signif(12345678, 4)
```

```
# [1] 12350000
```

```r
sin(pi / 2); cos(pi / 2)  # cos not exactly 0 (floating point)
```

```
# [1] 1
# [1] 6.123234e-17
```

Statistical Functions

Function	Definition
`max(x)`, `min(x)`, `range(x)`	max, min, c(min, max)
`sum(x)`, `prod(x)`	sum, product
`mean(x)`, `median(x)`	mean, median
`quantile(x, p)`	quantile at probability `p`
`var(x)`, `sd(x)`	sample variance, std deviation
`cor(x, y)`, `cov(x, y)`	correlation, covariance
`sort(x)`, `rev(x)`	sorted copy, reversed copy
`rank(x)`, `order(x)`	ranks, permutation to sort
`cumsum`, `cumprod`, `cummax`, `cummin`	cumulative operations
`diff(x)`	lagged differences
`pmax(x,y)`, `pmin(x,y)`	parallel (elementwise) max/min
`table(x)`	frequency table
`colMeans`, `colSums`	column means/sums
`rowMeans`, `rowSums`	row means/sums

Floating Point Surprises

```r
0.1 + 0.2 == 0.3               # FALSE!
```

```
# [1] FALSE
```

```r
all.equal(0.1 + 0.2, 0.3)      # TRUE (with tolerance)
```

```
# [1] TRUE
```

```r
isTRUE(all.equal(0.1 + 0.2, 0.3))  # safe for if()
```

```
# [1] TRUE
```

```r
# trunc vs floor for negative numbers
trunc(-3.7); floor(-3.7)      # toward 0 vs toward -Inf
```

```
# [1] -3
# [1] -4
```

Data Frames

What Is a Data Frame?

- A rectangular table: one row per observation, one column per variable.
- **Technically**: a named list of equal-length vectors (we'll revisit this in Lists).

```r
tg <- ToothGrowth    # built-in data; used throughout
head(tg, 4)
```

```
#    len supp dose
# 1  4.2   VC  0.5
# 2 11.5   VC  0.5
# 3  7.3   VC  0.5
# 4  5.8   VC  0.5
```

```r
# Each column is a vector; use $ to extract
tg$len[1:5]
```

```
# [1]  4.2 11.5  7.3  5.8  6.4
```

Data Frame Inspection

```r
str(tg)
```

```
# 'data.frame': 60 obs. of  3 variables:
#  $ len : num  4.2 11.5 7.3 5.8 6.4 10 11.2 11.2 5.2 7 ...
#  $ supp: Factor w/ 2 levels "OJ","VC": 2 2 2 2 2 2 2 2 2 2 ...
#  $ dose: num  0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 ...
```

```r
nrow(tg); ncol(tg); dim(tg); names(tg)
```

```
# [1] 60
# [1] 3
# [1] 60  3
# [1] "len"  "supp" "dose"
```

summary() — Quick Statistics

```r
summary(tg)
```

```
#       len        supp         dose      
#  Min.   : 4.20   OJ:30   Min.   :0.500  
#  1st Qu.:13.07   VC:30   1st Qu.:0.500  
#  Median :19.25           Median :1.000  
#  Mean   :18.81           Mean   :1.167  
#  3rd Qu.:25.27           3rd Qu.:2.000  
#  Max.   :33.90           Max.   :2.000  
```

Indexing: [row, col]

```r
tg[3, 1]              # row 3, column 1
```

```
# [1] 7.3
```

```r
tg$len[3]             # equivalent via $
```

```
# [1] 7.3
```

```r
(z <- tg[2:4, c(1, 3)])   # rows 2-4, columns 1 & 3
```

```
#    len dose
# 2 11.5  0.5
# 3  7.3  0.5
# 4  5.8  0.5
```

```r
head(tg[, c(1, 3)], 3)   # all rows, columns 1 & 3
```

```
#    len dose
# 1  4.2  0.5
# 2 11.5  0.5
# 3  7.3  0.5
```

```r
head(tg[, -2], 3)         # all rows, remove column 2
```

Creating Data Frames

```r
age <- c(55, 58, 45)
name <- c('Alice', 'Bill', 'Cathy')
(d <- data.frame(age, name))
```

```
#   age  name
# 1  55 Alice
# 2  58  Bill
# 3  45 Cathy
```

- Vectors of different lengths are not allowed!

```r
blood <- c('O', 'B', 'A', 'AB')
data.frame(age, name, blood)
```

```
# Error in data.frame(age, name, blood): arguments imply differing number of rows: 3, 4
```

```r
# with custom column names
data.frame(p1 = pred1, p2 = pred2)

# from a named list of vectors
l <- list(p1 = pred1, p2 = pred2)
as.data.frame(l)
```

```r
# from row data via rbind
r1 <- data.frame(a=1, b=2, c="X")
r2 <- data.frame(a=3, b=4, c="Y")
rbind(r1, r2)
```

Row Filtering with Logical Conditions

```r
tg_vc <- tg[tg$supp == 'VC', ]
tg_oj <- tg[tg$supp == 'OJ', ]
mean(tg_vc$len); mean(tg_oj$len)
```

```
# [1] 16.96333
```

```
# [1] 20.66333
```

```r
# AND condition
tg[tg$supp == 'OJ' & tg$len < 8.8, ]
```

```
#    len supp dose
# 37 8.2   OJ  0.5
```

```r
# OR condition
head(tg[tg$len > 28 | tg$dose == 1.0, ], 6)
```

```
#     len supp dose
# 11 16.5   VC    1
# 12 16.5   VC    1
# 13 15.2   VC    1
# 14 17.3   VC    1
# 15 22.5   VC    1
```

subset() — Cleaner Filtering

```r
# equivalent to tg[tg$supp == 'OJ' & tg$len > 20, ]
subset(tg, supp == 'OJ' & len > 20)
```

```
#     len supp dose
# 32 21.5   OJ  0.5
# 42 23.3   OJ  1.0
# 43 23.6   OJ  1.0
# 44 26.4   OJ  1.0
# 46 25.2   OJ  1.0
# 47 25.8   OJ  1.0
# 48 21.2   OJ  1.0
# 50 27.3   OJ  1.0
# 51 25.5   OJ  2.0
# 52 26.4   OJ  2.0
# 53 22.4   OJ  2.0
# 54 24.5   OJ  2.0
# 55 24.8   OJ  2.0
# 56 30.9   OJ  2.0
# 57 26.4   OJ  2.0
# 58 27.3   OJ  2.0
# 59 29.4   OJ  2.0
```

with() and within()

```r
# with() evaluates an expression in the data frame environment
with(tg, mean(len[supp == 'VC']))
```

```
# [1] 16.96333
```

```r
# within() modifies and returns the data frame
tg2 <- within(tg, {
  len_cm  <- len / 10
  log_len <- log(len)
})
head(tg2, 3)
```

```
#    len supp dose  log_len len_cm
# 1  4.2   VC  0.5 1.435085   0.42
# 2 11.5   VC  0.5 2.442347   1.15
# 3  7.3   VC  0.5 1.987874   0.73
```

Adding, Removing, Renaming Columns

```r
d <- data.frame(x = 1:3, y = c(10, 20, 30))
d$z <- d$x + d$y                # add column with $
d[['w']] <- c('a', 'b', 'c') # add column with [[]]
d
```

```
#   x  y  z w
# 1 1 10 11 a
# 2 2 20 22 b
# 3 3 30 33 c
```

```r
d$z <- NULL                     # remove column
names(d)[names(d) == 'w'] <- 'label'   # rename
d
```

```
#   x  y label
# 1 1 10     a
# 2 2 20     b
# 3 3 30     c
```

Sorting Data Frames

```r
d <- data.frame(name = c('Zara', 'Amy', 'Mike'),
                score = c(88, 95, 72))
d[order(d$score), ]                          # ascending
```

```
#   name score
# 3 Mike    72
# 1 Zara    88
# 2  Amy    95
```

```r
d[order(d$score, decreasing = TRUE), ]      # descending
```

```
#   name score
# 2  Amy    95
# 1 Zara    88
# 3 Mike    72
```

```r
# sort by multiple columns
d2 <- data.frame(g = c('A','B','A','B'), x = c(3,1,2,4))
d2[order(d2$g, d2$x), ]
```

```
#   g x
# 3 A 2
# 1 A 3
```

Handling NAs

```r
df <- data.frame(x = c(1, NA, 3, 4, 5), y = c(1, 2, NA, 4, 5))
na.omit(df)                          # remove rows with any NA
```

```
#   x y
# 1 1 1
# 4 4 4
# 5 5 5
```

```r
complete.cases(df)                    # logical: which rows complete?
```

```
# [1]  TRUE FALSE FALSE  TRUE  TRUE
```

```r
df[complete.cases(df), ]
```

```
#   x y
# 1 1 1
# 4 4 4
# 5 5 5
```

```r
df$x[is.na(df$x)] <- 0               # replace specific column NAs
df[is.na(df)] <- 0                   # replace all NAs
```

Combining Data Frames: cbind, rbind

```r
# column-bind (side by side) --- same number of rows
df1 <- data.frame(a = c(1, 2)); df2 <- data.frame(b = c(7, 8))
cbind(df1, df2)
```

```
#   a b
# 1 1 7
# 2 2 8
```

```r
# row-bind (stack) --- same column names
df1 <- data.frame(x = c("a", "a"), y = c(5, 6))
df2 <- data.frame(x = c("b", "b"), y = c(9, 10))
rbind(df1, df2)
```

```
#   x  y
# 1 a  5
# 2 a  6
# 3 b  9
# 4 b 10
```

The Split–Apply–Combine Paradigm

Step	Base R tool	What it does
Split	`split()`	partition data by factor → list
Apply	`lapply/sapply/tapply/…`	run function on each piece
Combine	`unsplit/rbind/do.call/…`	reassemble results
All-in-one	`aggregate()`	split + apply + combine in one call
Horizontal combine	`merge()`	join two tables by key

split() — Partition Data

```r
scores <- c(88, 92, 78, 95, 85, 70, 91, 83)
group  <- c('A','B','A','B','A','B','A','B')
(sg <- split(scores, group))
```

```
# $A
# [1] 88 78 85 91
#
# $B
# [1] 92 95 70 83
```

```r
sapply(sg, mean)          # per-group mean
```

```
#    A    B
# 85.5 85.0
```

```r
# split a data frame --- each element is a complete data frame
tg_split <- split(tg, tg$supp)
head(tg_split$OJ, 3)
```

```
#     len supp dose
# 31 15.2   OJ  0.5
# 32 21.5   OJ  0.5
# 33 17.6   OJ  0.5
```

split() — Multi-Factor and unsplit()

```r
tg_split2 <- split(tg, list(tg$supp, tg$dose))
names(tg_split2)
```

```
# [1] "OJ.0.5" "VC.0.5" "OJ.1"   "VC.1"   "OJ.2"   "VC.2"
```

```r
# unsplit: put pieces back in their original positions
sg <- split(1:8, rep(c('A','B'), 4))
sg$A <- sg$A * 10; sg$B <- sg$B * (-1)
unsplit(sg, rep(c('A','B'), 4))
```

```
# [1] 10 -2 30 -4 50 -6 70 -8
```

The split + lapply + rbind Pipeline

```r
# add a z-score column within each group
standardize <- function(df) {
  df$len_z <- (df$len - mean(df$len)) / sd(df$len)
  df
}
parts <- split(tg, tg$supp)
parts <- lapply(parts, standardize)
tg2 <- do.call(rbind, parts)
head(tg2[order(as.numeric(rownames(tg2))), ], 4)
```

```
#        len supp dose      len_z
# OJ.31 15.2   OJ  0.5 -0.8270809
# OJ.32 21.5   OJ  0.5  0.1266610
# OJ.33 17.6   OJ  0.5 -0.4637507
# OJ.34  9.7   OJ  0.5 -1.6597127
```

aggregate() — Group-wise Summaries

```r
# formula syntax: response ~ grouping
aggregate(len ~ supp, data = tg, FUN = mean)
```

```
#   supp      len
# 1   OJ 20.66333
# 2   VC 16.96333
```

```r
aggregate(len ~ supp + dose, data = tg, FUN = mean)
```

```
#   supp dose   len
# 1   OJ  0.5 13.23
# 2   VC  0.5  7.98
# 3   OJ  1.0 22.70
# 4   VC  1.0 16.77
# 5   OJ  2.0 26.06
# 6   VC  2.0 26.14
```

aggregate() — Custom Summary and Multiple Responses

```r
# custom function: result is a matrix column!
aggregate(len ~ supp, data = tg,
          FUN = function(x) c(mean = mean(x), sd = sd(x)))
```

```
#   supp  len.mean    len.sd
# 1   OJ 20.663333  6.605561
# 2   VC 16.963333  8.266029
```

```r
# multiple response columns with cbind()
aggregate(cbind(mpg, hp, wt) ~ cyl, data = mtcars, FUN = mean)
```

```
#   cyl      mpg        hp       wt
# 1   4 26.66364  82.63636 2.285727
# 2   6 19.74286 122.28571 3.117143
# 3   8 15.10000 209.21429 3.999214
```

aggregate() — Expand Matrix Column and . Shorthand

```r
# expand matrix column into proper data frame
res <- aggregate(len ~ supp, data = tg,
                 FUN = function(x) c(m = mean(x), s = sd(x)))
cbind(res[1], as.data.frame(res$len))
```

```
#   supp        m        s
# 1   OJ 20.66333 6.605561
# 2   VC 16.96333 8.266029
```

```r
# . means "all other columns"
aggregate(. ~ cyl, data = mtcars[, c('mpg','hp','wt','cyl')],
          FUN = mean)
```

```
#   cyl      mpg        hp       wt
# 1   4 26.66364  82.63636 2.285727
# 2   6 19.74286 122.28571 3.117143
# 3   8 15.10000 209.21429 3.999214
```

merge() — Joining Data Frames

```r
customers <- data.frame(id = 1:4,
  name = c('Alice', 'Bob', 'Cathy', 'Dan'))
orders <- data.frame(cust_id = c(2, 3, 3, 1, 5),
  product = c('pen','book','ink','paper','tape'),
  amount = c(12, 35, 8, 20, 5))
# inner join (default): only matched rows
merge(customers, orders, by.x = 'id', by.y = 'cust_id')
```

```
#   id  name product amount
# 1  1 Alice   paper     20
# 2  2   Bob     pen     12
# 3  3 Cathy    book     35
# 4  3 Cathy     ink      8
```

merge() — Join Types

```r
# left join: keep all customers (NA for no match)
merge(customers, orders, by.x='id', by.y='cust_id', all.x=TRUE)
```

```
#   id  name product amount
# 1  1 Alice   paper     20
# 2  2   Bob     pen     12
# 3  3 Cathy    book     35
# 4  3 Cathy     ink      8
# 5  4   Dan    <NA>     NA
```

SQL	merge() arguments	Keeps
INNER JOIN	default (all=FALSE)	matched rows only
LEFT JOIN	all.x = TRUE	all rows in x
RIGHT JOIN	all.y = TRUE	all rows in y
FULL OUTER JOIN	all = TRUE	all rows in both

merge() — Multi-Column Keys and Suffixes

```r
# multi-column key
sales <- data.frame(year=c(2023,2023,2024), qtr=c(1,2,1), rev=c(100,120,110))
targets <- data.frame(year=c(2023,2023,2024), qtr=c(1,2,1), target=c(95,115,105))
merge(sales, targets, by = c('year', 'qtr'))
```

```
#   year qtr rev target
# 1 2023   1 100     95
# 2 2023   2 120    115
# 3 2024   1 110    105
```

```r
# duplicate column names get suffixes
df1 <- data.frame(id = 1:3, value = c(10, 20, 30))
df2 <- data.frame(id = 1:3, value = c(100, 200, 300))
merge(df1, df2, by = 'id', suffixes = c('_old', '_new'))
```

```
#   id value_old value_new
# 1  1        10       100
# 2  2        20       200
# 3  3        30       300
```

Lists

What Is a List?

- A **generic vector**: elements can be of **different types and lengths**.

```
vn <- c(1, 1.2, 2.3, 3.4, 4.5)
vb <- c(TRUE, TRUE, FALSE)
vc <- c('limestone', 'marl', 'oolite', 'CaCO3')
```

```
data.frame(vn, vb, vc)   # ERROR: different lengths
```

```
# Error in data.frame(vn, vb, vc): arguments imply differing number of rows: 5, 3, 4
```

Creating Lists

```r
# unnamed list
l <- list(3.14, 'Moe', c(1, 3, 5, 2), mean)
str(l)
```

```
# List of 4
#  $ : num 3.14
#  $ : chr "Moe"
#  $ : num [1:4] 1 3 5 2
#  $ :function (x, ...)
```

```r
# named list + incremental construction
l <- list(max = 5, right = 0.6, mean = 20)
l <- list(); l$left <- 0.1; l$right <- 0.52; l$mid <- 0.33
l
```

```
# $left
# [1] 0.1
#
# $right
# [1] 0.52
#
# $mid
```

[[vs [— The Most Important Distinction

```r
l <- list(a = 1:3, b = 'hello',
          c = TRUE)
l[[1]]      # the element itself
```

```
# [1] 1 2 3
```

```r
l[1]        # a sub-list
```

```
# $a
# [1] 1 2 3
```

```r
class(l[[1]]); class(l[1])
```

```
# [1] "integer"
# [1] "list"
```

```r
l$a   # same as l[['a']]
```

```
# [1] 1 2 3
```

```r
l$b
```

```
# [1] "hello"
```

Analogy: [[reaches into the box and pulls out the item; [selects boxes (still wrapped).

Modifying, Removing, Concatenating

```r
l <- list(x = 1, y = 2, z = 3)
l$x <- 100; l[['y']] <- 200; l$w <- 'new'   # modify / add
l[c('x', 'w')] <- NULL; l                     # remove
```

```
# $y
# [1] 200
#
# $z
# [1] 3
```

```r
l1 <- list(a = 1, b = 2); l2 <- list(c = 3, d = 4)
c(l1, l2)    # concatenate lists
```

```
# $a
# [1] 1
#
# $b
# [1] 2
#
# $c
# [1] 3
#
```

unlist() — Flatten a List

```r
iq <- list(100, 120, 140, 112, 105)
mean(unlist(iq))       # must unlist before mean()
```

```
# [1] 115.4
```

```r
l <- list(a = 1:3, b = 4:5)
unlist(l)                          # names preserved with prefix
```

```
# a1 a2 a3 b1 b2
#  1  2  3  4  5
```

```r
unlist(l, use.names = FALSE)     # drop names
```

```
# [1] 1 2 3 4 5
```

split() — Split a Vector into a List

```r
(mtl <- split(mtcars$mpg, mtcars$cyl))
```

```
# $`4`
#  [1] 22.8 24.4 22.8 32.4 30.4 33.9 21.5 27.3 26.0 30.4 21.4
#
# $`6`
# [1] 21.0 21.0 21.4 18.1 19.2 17.8 19.7
#
# $`8`
#  [1] 18.7 14.3 16.4 17.3 15.2 10.4 10.4 14.7 15.5 15.2 13.3 19.2 15.8 15.0
```

```r
mtl$`6`; mtl[[2]]   # access by name or position
```

```
# [1] 21.0 21.0 21.4 18.1 19.2 17.8 19.7
# [1] 21.0 21.0 21.4 18.1 19.2 17.8 19.7
```

- `tapply(x, f, FUN)` is equivalent to `sapply(split(x, f), FUN)`

A Data Frame Is a List

```r
df <- data.frame(x = 1:3, y = c('a', 'b', 'c'))
is.list(df); typeof(df)
```

```
# [1] TRUE
# [1] "list"
```

```r
df$x; df[['x']]     # extracts column vector (like [[)
```

```
# [1] 1 2 3
# [1] 1 2 3
```

```r
df['x']              # returns 1-column data.frame (like [)
```

```
#   x
# 1 1
# 2 2
# 3 3
```

```r
# split() a data frame -> list of data frames
str(split(tg, tg$supp))
```

```
# List of 2
#  $ OJ:'data.frame':   30 obs. of  3 variables:
```

Conversions Between Data Structures

From	To	How	Note
vector	list	`as.list(v)`	Don't use `list(v)`!
vector	matrix	`matrix(v, n, m)`	
vector	data.frame	`as.data.frame(v)`	
list	vector	`unlist(l)`	Not `as.vector`
list	data.frame	`as.data.frame(l)`	Equal-length only
matrix	vector	`as.vector(m)`	
matrix	data.frame	`as.data.frame(m)`	
data.frame	list	`as.list(df)`	Removes class attr
data.frame	matrix	`as.matrix(df)`	Coercion!

Matrices

Creating Matrices

```r
v <- 1:6; matrix(v, nrow = 2, ncol = 3)
```

```
#      [,1] [,2] [,3]
# [1,]    1    3    5
# [2,]    2    4    6
```

```r
# named rows and columns
m <- matrix(c(1, .56, .38, .56, 1, .44, .38, .44, 1), 3, 3)
colnames(m) <- rownames(m) <- c('AAPL', 'MSFT', 'GOOG')
m; m['MSFT', 'GOOG']
```

```
#      AAPL MSFT GOOG
# AAPL 1.00 0.56 0.38
# MSFT 0.56 1.00 0.44
# GOOG 0.38 0.44 1.00
```

```
# [1] 0.44
```

Matrix Operations

```r
A <- matrix(1:4, 2, 2); B <- matrix(5:8, 2, 2)
A * B        # element-wise
```

```
#      [,1] [,2]
# [1,]    5   21
# [2,]   12   32
```

```r
A %*% B      # matrix multiplication
```

```
#      [,1] [,2]
# [1,]   23   31
# [2,]   34   46
```

```r
t(A); det(A); solve(A)   # transpose, determinant, inverse
```

```
#      [,1] [,2]
# [1,]    1    2
# [2,]    3    4
```

```
# [1] -2
```

```
#      [,1] [,2]
# [1,]   -2  1.5
# [2,]    1 -0.5
```

Matrix Functions Summary

```r
nrow(m); ncol(m); dim(m)
diag(m)                    # diagonal elements
diag(3)                    # 3x3 identity matrix
crossprod(A, B)            # t(A) %*% B (faster)
tcrossprod(A, B)           # A %*% t(B)
outer(1:3, 1:3, '+')      # outer product
eigen(A)                   # eigenvalue decomposition
svd(A)                     # SVD
qr(A)                      # QR decomposition
```

Programming

Defining Functions

```r
function_name <- function(arg1, arg2 = default_value, ...) {
    # code
    return(some_object)
}
```

```r
# functions can read global variables
y_squared <- function() { return(y^2) }
y <- 2; y_squared()
```

```
# [1] 4
```

```r
# but local assignment doesn't change globals
add_to_y <- function(n) { y <- y + n }
y <- 1; add_to_y(2); y      # y is still 1
```

```
# [1] 1
```

```r
# <<- modifies globals (use sparingly!)
add_to_y_global <- function(n) { y <<- y + n }
y <- 1; add_to_y_global(2); y   # y is now 3
```

```
# [1] 3
```

`return()` and Implicit Return

```r
avg <- function(x) { return(sum(x) / length(x)) }
avg(c(2, 3, 6))
```

```
# [1] 3.666667
```

```r
avg(c(TRUE, FALSE, TRUE))   # TRUE=1, FALSE=0
```

```
# [1] 0.6666667
```

```r
# implicit return: the last evaluated expression is returned
avg_bad <- function(x) { r <- sum(x) / length(x) }
avg_ok  <- function(x) { sum(x) / length(x) }
avg_bad(1:10)   # silent! (assignment returns invisibly)
avg_ok(1:10)    # returns result
```

```
# [1] 5.5
```

Default Arguments and Named Arguments

```r
power_n <- function(x, n = 2) { return(x^n) }
power_n(3)              # default n = 2
```

```
# [1] 9
```

```r
power_n(3, 3)           # supplied n = 3
```

```
# [1] 27
```

```r
power_n(x = 5, n = 2)  # named arguments
```

```
# [1] 25
```

```r
power_n(n = 2, x = 5)  # order doesn't matter with names
```

```
# [1] 25
```

Functions as Arguments and ...

```r
apply_to_first2 <- function(x, func, ...) {
    func(x[1:2], ...)
}
x <- c(NA, 2, 3)
apply_to_first2(x, sum, na.rm = TRUE)
```

```
# [1] 2
```

Anonymous Functions and \(x) Shorthand

```r
sapply(1:5, function(x) x^2 + 1)
```

```
# [1]  2  5 10 17 26
```

```r
sapply(1:5, \(x) x^2 + 1)   # R 4.1+ shorthand
```

```
# [1]  2  5 10 17 26
```

```r
# useful in apply family
l <- list(a = 1:5, b = 6:10)
lapply(l, \(v) c(mean = mean(v), sd = sd(v)))
```

```
# $a
#     mean       sd
# 3.000000 1.581139
#
# $b
#     mean       sd
# 8.000000 1.581139
```

Lexical Scoping Rule

When R encounters a free variable inside a function, it searches the **environment where the function was defined** (not where it was called):

```
x <- 10
f <- function() x          # f is born in global env
g <- function() {
  x <- 999
  f()                       # calls f, but f ignores this x
}
g()                         # 10, not 999
```

```
# [1] 10
```

- **Lexical** (static): determined by where the code is **written**
- R, Python, JavaScript all use lexical scoping

What Is a Closure?

***Closure** = a function **bundled together** with the environment in which it was created.*

When function A returns function B, B retains a **live reference** to A's local environment — even after A has finished executing.

```
power_factory <- function(n) {
  function(x) x^n              # captures n
}
square <- power_factory(2)
cube   <- power_factory(3)
square(5); cube(5)
```

```
# [1] 25
# [1] 125
```

```
# each call creates a new, independent environment
environment(square)$n; environment(cube)$n
```

```
# [1] 2
# [1] 3
```

Inspecting Function Internals

R does not distinguish "closure" as a special type. Every function stores `formals`, `body`, and `environment`:

```r
f <- function(x) x + 1
typeof(f)          # "closure"
```

```
# [1] "closure"
```

```r
formals(f); body(f); environment(f)
```

```
# $x
# x + 1
# <environment: R_GlobalEnv>
```

<- vs <<- and Mutable State

- <- assigns **only** in the current (local) environment
- <<- walks **up** the enclosing environment chain to modify an existing variable

```
make_counter <- function() {
  n <- 0
  list(
    increment = function() { n <<- n + 1; n },
    get       = function() n,
    reset     = function() { n <<- 0 }
  )
}
ctr <- make_counter()
ctr$increment(); ctr$increment(); ctr$get()
```

```
# [1] 1
# [1] 2
# [1] 2
```

```
ctr$reset(); ctr$get()
```

```
# [1] 0
```

Independent Instances and `local()`

```r
ctr1 <- make_counter(); ctr2 <- make_counter()
ctr1$increment(); ctr1$increment(); ctr2$increment()
```

```
# [1] 1

# [1] 2

# [1] 1
```

```r
cat("ctr1:", ctr1$get(), " ctr2:", ctr2$get(), "\n")
```

```
# ctr1: 2  ctr2: 1
```

```r
# local() creates a single-instance closure
counter <- local({
  n <- 0
  list(
    increment = function() { n <<- n + 1; n },
    get       = function() n,
    reset     = function() { n <<- 0 })
})
counter$increment(); counter$increment(); counter$get()
```

```
# [1] 1
```

Classic Pitfall: Loop Capture

```r
funs <- list()
for (i in 1:5) funs[[i]] <- function() i
sapply(funs, \(f) f())  # all 5!
```

```
# [1] 5 5 5 5 5
```

All five functions share the **same** environment; after the loop, `i = 5`.

```r
# Fix 1: force() defeats lazy evaluation
make_fun <- function(val) { force(val); function() val }
funs <- list()
for (i in 1:5) funs[[i]] <- make_fun(i)
sapply(funs, \(f) f())
```

```
# [1] 1 2 3 4 5
```

```r
# Fix 2 (recommended): lapply creates separate environments
funs <- lapply(1:5, \(i) function() i)
sapply(funs, \(f) f())
```

```
# [1] 1 2 3 4 5
```

Base R Closures: ecdf, approxfun, Negate

```r
x <- c(1, 3, 3, 5, 7, 9); Fn <- ecdf(x)
Fn(4); Fn(7)        # P(X<=4)=3/6; P(X<=7)=5/6
```

```
# [1] 0.5
# [1] 0.8333333
```

```r
typeof(Fn)           # "closure"
```

```
# [1] "closure"
```

```r
xp <- 1:5; yp <- c(2, 5, 3, 8, 6)
linear_interp <- approxfun(xp, yp)
linear_interp(2.5)   # returns a closure over the knot data
```

```
# [1] 4
```

```r
is_even <- function(x) x %% 2 == 0
is_odd  <- Negate(is_even)          # closure combinator
Filter(is_odd, 1:10)
```

```
# [1] 1 3 5 7 9
```

Decorator and Iterator Patterns

```r
with_logging <- function(f, fname = deparse(substitute(f))) {
  call_count <- 0L
  function(...) {
    call_count <<- call_count + 1L
    cat(sprintf("[%s] call #%d\n", fname, call_count))
    f(...)
  }
}
logged_mean <- with_logging(mean)
logged_mean(1:10); logged_mean(c(3, NA, 7), na.rm = TRUE)
```

```
# [mean] call #1
# [1] 5.5
# [mean] call #2
# [1] 5
```

Moving Average and Fibonacci Iterator

```r
make_ema <- function(alpha = 0.3) {
  value <- NULL
  function(x) {
    if (is.null(value)) { value <<- x }
    else { value <<- alpha * x + (1 - alpha) * value }
    value
  }
}
ema <- make_ema(0.2)
sapply(c(100, 102, 101, 105, 103, 107, 106), ema)
```

```
# [1] 100.0000 100.4000 100.5200 101.4160 101.7328 102.7862 103.4290
```

```r
make_fib_iterator <- function() {
  a <- 0; b <- 1
  list(next_val = function() {
    val <- a; new <- a + b; a <<- b; b <<- new; val
  }, reset = function() { a <<- 0; b <<- 1 })
}
fib_it <- make_fib_iterator()
replicate(10, fib_it$next_val())
```

if / else

```r
reciprocal <- function(x) {
  if (is.na(x)) {
    cat("Error! Division by NA.\n")
  } else if (is.infinite(x)) {
    cat("Error! Division by infinity.\n")
  } else if (x == 0) {
    cat("Error! Division by zero.\n")
  } else { 1 / x }
}
reciprocal(2); reciprocal(0); reciprocal(NA)
```

```
# [1] 0.5
# Error! Division by zero.
# Error! Division by NA.
```

ifelse() — Vectorized Conditional

```r
x <- c(-1, 2, -100, 20, 50)
ifelse(x > 0, 'positive', 'negative')
```

```
# [1] "negative" "positive" "negative" "positive" "positive"
```

```r
# nested ifelse
v <- c(20, 18, 45, 33, 25, 9, 17, 55, 69, 81, 44, 32)
ifelse(v >= 20 & v < 40, 'young adult',
ifelse(v >= 40 & v < 60, 'middle age', 'other'))
```

```
#  [1] "young adult" "other"       "middle age"  "young adult" "young adult"
#  [6] "other"       "other"       "middle age"  "other"       "other"
# [11] "middle age"  "young adult"
```

switch()

```r
day_type <- function(day) {
  switch(day,
    Monday=, Tuesday=, Wednesday=,
    Thursday=, Friday= "Weekday",
    Saturday=, Sunday= "Weekend",
    "Unknown")
}
day_type("Monday"); day_type("Sunday"); day_type("Holiday")
```

```
# [1] "Weekday"
# [1] "Weekend"
# [1] "Unknown"
```

&& vs & (Short-Circuit Evaluation)

```r
# first, ensure 'aa' does not exist from earlier chunks
if (exists("aa")) rm(aa)
```

```r
# && is short-circuited: stops at first FALSE
if (exists("aa") && aa > 0) {
    cat("aa exists and is positive.\n")
} else {
    cat("aa doesn't exist or is negative.\n")
}
```

```
# aa doesn't exist or is negative.
```

```r
# & evaluates BOTH sides -- error because aa doesn't exist!
if (exists("aa") & aa > 0) {
    cat("aa exists and is positive.\n")
} else {
    cat("aa doesn't exist or is negative.\n")
}
```

```
# Error: object 'aa' not found
```

- **Rule**: Use `&&` / `||` in `if` (scalar); use `&` / `|` for vectors.

`if (FALSE) { }` — Code Block Toggle

```r
# wrap example/test code that shouldn't run
if (FALSE) {
  # these lines are "commented out" but remain valid R code
  book(date = "2026/04/02", headless = FALSE)
  cancel(order_id = "01901879")
}
```

- Unlike # comments, the code inside is still checked by R's parser

for Loop

```r
# sum 1 to 10000
ans <- 0
for (i in 1:10000) { ans <- ans + i }
ans
```

```
# [1] 50005000
```

```r
# sum of min(2^k, k^4) for k = 1..100
ans <- 0
for (k in 1:100) { ans <- ans + min(2^k, k^4) }
ans
```

```
# [1] 2050220551
```

```r
# vectorized equivalent (much faster)
v <- 1:100; sum(pmin(2^v, v^4))
```

```
# [1] 2050220551
```

while, repeat, break, next

```r
# while: find smallest n such that sum(1:n) > 1000
n <- 0; s <- 0
while (s <= 1000) { n <- n + 1; s <- s + n }
cat("n =", n, ", sum =", s, "\n")
```

```
# n = 45 , sum = 1035
```

```r
# repeat + break
n <- 1
repeat { if (n^2 > 100) break; n <- n + 1 }
cat("First n with n^2 > 100:", n, "\n")
```

```
# First n with n^2 > 100: 11
```

```r
# next: skip iterations
for (i in 1:10) {
  if (i %% 3 == 0) next    # skip multiples of 3
  cat(i, '')
}
```

```
# 1 2 4 5 7 8 10
```

Vectorization vs Loops: Speed Comparison

```r
my_sum <- function(x) {
    ans <- 0
    for (i in seq_along(x)) { ans <- ans + x[i] }
    ans
}
v <- 1:1000000; system.time(my_sum(v)); system.time(sum(v))
```

```
#    user  system elapsed
#   0.032   0.000   0.032
```

```
#    user  system elapsed
#       0       0       0
```

```r
# pre-allocate vs grow-in-loop vs vectorized
n <- 1e5
system.time({v <- c(); for (i in 1:n) v <- c(v, i^2)})
```

```
#    user  system elapsed
#   6.095   0.107   6.203
```

```r
system.time({v <- numeric(n); for (i in 1:n) v[i] <- i^2})
```

```
#    user  system elapsed
```

Native Pipe |> (R 4.1+)

```r
# without pipe: nested calls are hard to read
round(mean(abs(rnorm(100))), 3)
```

```
# [1] 0.809
```

```r
# with pipe: read left to right
set.seed(1)
rnorm(100) |> abs() |> mean() |> round(3)
```

```
# [1] 0.717
```

```r
# pipe with data frame operations
mtcars |> subset(cyl == 6) |> head(3)
```

```
#                 mpg cyl disp  hp drat    wt  qsec vs am gear carb
# Mazda RX4      21.0   6  160 110 3.90 2.620 16.46  0  1    4    4
# Mazda RX4 Wag  21.0   6  160 110 3.90 2.875 17.02  0  1    4    4
# Hornet 4 Drive 21.4   6  258 110 3.08 3.215 19.44  1  0    3    1
```

```r
# use _ as placeholder for non-first argument (R 4.2+)
mtcars |> lm(mpg ~ wt, data = _) |> summary()
```

The Apply Family

apply() — Over Matrix / Data Frame Margins

```r
m <- matrix(1:12, nrow = 3,
            dimnames = list(paste0('r',1:3), paste0('c',1:4)))
m
```

```
#    c1 c2 c3 c4
# r1  1  4  7 10
# r2  2  5  8 11
# r3  3  6  9 12
```

```r
apply(m, 1, sum)     # MARGIN=1: rows
```

```
# r1 r2 r3
# 22 26 30
```

```r
apply(m, 2, sum)     # MARGIN=2: columns
```

```
# c1 c2 c3 c4
#  6 15 24 33
```

```r
# when FUN returns a vector, result is k x n matrix
apply(m, 2, range)
```

```
#      c1 c2 c3 c4
# [1,]  1  4  7 10
```

apply() on Data Frames — Beware of Coercion!

```r
df <- data.frame(x = 1:3, y = c('a','b','c'), z = 4:6)
apply(df, 2, class)       # all "character"!
```

```
#           x           y           z
# "character" "character" "character"
```

- apply() converts data frames to matrices first — **all columns become character** if any is character. Use sapply/lapply for mixed types.

```r
# faster alternatives for row/column sums/means:
colSums(m); rowSums(m); colMeans(m); rowMeans(m)
```

```
# c1 c2 c3 c4
#  6 15 24 33

# r1 r2 r3
# 22 26 30

# c1 c2 c3 c4
#  2  5  8 11

#  r1  r2  r3
# 5.5 6.5 7.5
```

lapply() and sapply()

```r
x <- list(a = 1:5, b = rnorm(10), c = c(TRUE, FALSE, TRUE))
lapply(x, mean)      # always returns a list
```

```
# $a
# [1] 3
#
# $b
# [1] 0.3474802
#
# $c
# [1] 0.6666667
```

```r
sapply(x, mean)      # simplifies to named vector
```

```
#         a         b         c
# 3.0000000 0.3474802 0.6666667
```

```r
# sapply can be unpredictable; vapply is safer
vapply(x, mean, numeric(1))
```

```
#         a         b         c
# 3.0000000 0.3474802 0.6666667
```

sapply() Is Unpredictable — Use vapply() in Production

```r
# when all lengths equal: matrix. When they differ: list!
sapply(list(a = 1:3, b = 4:6), identity)
```

```
#      a b
# [1,] 1 4
# [2,] 2 5
# [3,] 3 6
```

```r
sapply(list(a = 1:3, b = 4:5), identity)
```

```
# $a
# [1] 1 2 3
#
# $b
# [1] 4 5
```

```r
# vapply: explicit type template; error if mismatch
vapply(iris[,1:4], median, numeric(1))
```

```
# Sepal.Length  Sepal.Width Petal.Length  Petal.Width
#         5.80         3.00         4.35         1.30
```

tapply() — Apply by Groups

```r
# mean mpg by cylinder count
tapply(mtcars$mpg, mtcars$cyl, mean)
```

```
#        4        6        8
# 26.66364 19.74286 15.10000
```

```r
# two-way grouping -> matrix
tapply(mtcars$mpg, list(mtcars$cyl, mtcars$am), mean)
```

```
#        0        1
# 4 22.900 28.07500
# 6 19.125 20.56667
# 8 15.050 15.40000
```

- `tapply(x, f, FUN)` $\equiv$ `sapply(split(x, f), FUN)`

mapply() and Map()

```r
# apply to corresponding elements of multiple vectors
mapply(function(x, y) x + y, 1:5, 10:14)
```

```
# [1] 11 13 15 17 19
```

```r
mapply(rep, 1:3, 3:1)   # variable-length results
```

```
# [[1]]
# [1] 1 1 1
#
# [[2]]
# [1] 2 2
#
# [[3]]
# [1] 3
```

```r
# Map: always returns a list (no simplification)
Map('+', 1:3, 10:12)
```

```
# [[1]]
# [1] 11
#
# [[2]]
```

rapply() — Recursive Apply

```r
nested <- list(a = 1:3,
  b = list(c = 4:6, d = list(e = 7:9, f = 'text')))
rapply(nested, sum, classes = 'integer')         # unlist
```

```
#     a   b.c b.d.e
#     6    15    24
```

```r
rapply(nested, sum, classes = 'integer', how = 'list')
```

```
# $a
# [1] 6
#
# $b
# $b$c
# [1] 15
#
# $b$d
# $b$d$e
# [1] 24
#
# $b$d$f
# NULL
```

Reduce() — Sequential Accumulation

```r
Reduce('+', 1:5)                          # 15
```

```
# [1] 15
```

```r
Reduce('+', 1:5, accumulate = TRUE)       # running total
```

```
# [1]  1  3  6 10 15
```

```r
# merge multiple data frames
dfs <- list(data.frame(id=1:3, a=10:12),
            data.frame(id=2:4, b=20:22),
            data.frame(id=1:3, c=30:32))
Reduce(\(x, y) merge(x, y, by='id', all=TRUE), dfs)
```

```
#   id  a  b  c
# 1  1 10 NA 30
# 2  2 11 20 31
# 3  3 12 21 32
# 4  4 NA 22 NA
```

Filter(), Find(), Position(), Negate()

```r
x <- list(1:3, "hello", 4:6, TRUE, NULL)
Filter(is.numeric, x)          # keep numeric elements
```

```
# [[1]]
# [1] 1 2 3
#
# [[2]]
# [1] 4 5 6
```

```r
Find(is.character, x)          # first character element
```

```
# [1] "hello"
```

```r
Position(is.character, x)      # its position
```

```
# [1] 2
```

```r
Filter(\(x) x %% 15 == 0, 1:50)   # multiples of 15
```

```
# [1] 15 30 45
```

```r
# Find/Position with direction
Find(\(x) x > 5, c(2, 8, 3, 9, 1), right = TRUE)
```

Decision Table: Choosing the Right Tool

Function	Input	Output	Use case
`apply`	matrix/array	vector/matrix	row/column ops
`lapply`	vector/list	list	general iteration
`sapply`	vector/list	simplified	interactive only
`vapply`	vector/list	typed output	production code
`tapply`	vector + factor	array	group summaries
`mapply/Map`	multiple vectors	simplified/list	parallel iteration
`rapply`	nested list	various	recursive ops
`Reduce`	vector/list	single value	sequential fold
`Filter`	vector/list	subset	keep matching
`Find/Position`	vector/list	one element	first match
`Negate`	function	function	invert predicate

Common Equivalences

```
tapply(x, f, FUN)          <==>  sapply(split(x, f), FUN)
sapply(X, FUN)             <==>  unlist(lapply(X, FUN))   # scalar FUN
mapply(FUN, x, y)          <==>  sapply(seq_along(x),
                                   \(i) FUN(x[i], y[i]))
Map(FUN, x, y)             <==>  mapply(FUN, x, y, SIMPLIFY = FALSE)
Reduce('+', 1:5)           <==>  ((((1+2)+3)+4)+5)
Filter(f, x)               <==>  x[sapply(x, f)]
aggregate(y~g, d, mean)    <==>  tapply(d$y, d$g, mean)  # approx
```

String Operations

Basic String Functions

```r
nchar('hello world')
```

```
# [1] 11
```

```r
toupper('hello'); tolower('HELLO')
```

```
# [1] "HELLO"
# [1] "hello"
```

```r
trimws('  hello  '); trimws('  hello  ', which = 'left')
```

```
# [1] "hello"
# [1] "hello  "
```

```r
substr('abcdefgh', 3, 5)
```

```
# [1] "cde"
```

```r
startsWith('hello world', 'hello')
```

```
# [1] TRUE
```

```r
endsWith('hello world', 'world')
```

```
# [1] TRUE
```

paste() and paste0()

```r
paste('a', 'b', 'c', sep = '-')
```

```
# [1] "a-b-c"
```

```r
paste0('item', 1:5)
```

```
# [1] "item1" "item2" "item3" "item4" "item5"
```

```r
(labels <- paste(c('a', 'b'), 1:6, sep = '-'))
```

```
# [1] "a-1" "b-2" "a-3" "b-4" "a-5" "b-6"
```

```r
# collapse: join multiple strings into one
paste('item', 1:5, sep = '_', collapse = ', ')
```

```
# [1] "item_1, item_2, item_3, item_4, item_5"
```

sprintf() — Formatted Strings

```r
# %d: integer; %.2f: 2 decimal places; %s: string
sprintf('Name: %s, Age: %d, Score: %.2f', 'Alice', 25, 95.678)
```

```
# [1] "Name: Alice, Age: 25, Score: 95.68"
```

```r
sprintf('%05d', 42)          # zero-padded
```

```
# [1] "00042"
```

```r
sprintf('%.2f%%', 12.345)    # percentage
```

```
# [1] "12.35%"
```

```r
sprintf('%-10s|', 'left')    # left-align
```

```
# [1] "left      |"
```

cat() — Print to Console

```r
v <- c(20, 30, 3 * pi)
cat("The content of v is:", v, "...\n")
```

```
# The content of v is: 20 30 9.424778 ...
```

```r
for (i in 50 * (0:2)) {
  cat(sprintf('exp(sin(%3d)) = %.6f', i, exp(sin(i))), '\n')
}
```

```
# exp(sin(  0)) = 1.000000
# exp(sin( 50)) = 0.769223
# exp(sin(100)) = 0.602682
```

strsplit() — Split Strings

```r
strsplit('a.b.c.d', split = '\\.')
```

```
# [[1]]
# [1] "a" "b" "c" "d"
```

```r
strsplit('hello world foo', split = ' ')
```

```
# [[1]]
# [1] "hello" "world" "foo"
```

```r
strsplit(c('a-b', 'c-d-e'), split = '-')
```

```
# [[1]]
# [1] "a" "b"
#
# [[2]]
# [1] "c" "d" "e"
```

```r
strsplit('hello', split = '')   # every character
```

```
# [[1]]
# [1] "h" "e" "l" "l" "o"
```

grep(), grepl() — Pattern Matching

```r
x <- c('apple', 'banana', 'cherry', 'apricot', 'blueberry')
grep('an', x)                                    # indices
```

```
# [1] 2
```

```r
grep('an', x, value = TRUE)                     # values
```

```
# [1] "banana"
```

```r
grepl('an', x)                                   # logical
```

```
# [1] FALSE  TRUE FALSE FALSE FALSE
```

```r
grep('APPLE', x, ignore.case = TRUE, value = TRUE)
```

```
# [1] "apple"
```

sub(), gsub() — Pattern Replacement

```r
x <- 'Hello World Hello'
sub('Hello', 'Hi', x)      # replace FIRST match
```

```
# [1] "Hi World Hello"
```

```r
gsub('Hello', 'Hi', x)     # replace ALL matches
```

```
# [1] "Hi World Hi"
```

```r
gsub('[0-9]', '', 'abc123def456')      # remove all digits
```

```
# [1] "abcdef"
```

```r
gsub(' +', ' ', 'too   many    spaces')  # compress spaces
```

```
# [1] "too many spaces"
```

regexpr(), gregexpr(), regmatches()

```r
x <- 'Phone: 02-1234-5678, Fax: 04-8765-4321'
m <- gregexpr('[0-9]+-[0-9]+-[0-9]+', x)
regmatches(x, m)    # extract all matches
```

```
# [[1]]
# [1] "02-1234-5678" "04-8765-4321"
```

Regular Expressions

What Are Regular Expressions?

- Patterns for matching text. Used in `grep`, `grepl`, `sub`, `gsub`, `regexpr`, `strsplit`, etc.
- In R strings, backslash must be **doubled**: write `\\d` for the regex `\d`.

Basic Metacharacters

Pattern	Meaning
`.`	any single character (except newline)
`^`	start of string
`$`	end of string
`*`	0 or more of preceding
`+`	1 or more of preceding
`?`	0 or 1 of preceding
`{n}`, `{n,m}`	exactly `n`; between `n` and `m`
`|`	alternation (or)
`()`	grouping and capture
`[]`	character class
`\`	escape next character

Character Classes

Class	Meaning
`[abc]`	matches a, b, or c
`[^abc]`	matches anything except a, b, c
`[a-z]`, `[A-Z]`, `[0-9]`	ranges
`\d` / `\D`	digit / non-digit
`\w` / `\W`	word char / non-word char
`\s` / `\S`	whitespace / non-whitespace
`\b`	word boundary

```r
x <- c('hello123', 'world', '456', 'test!', 'a1b2')
grep('[0-9]', x, value = TRUE)       # contains a digit
```

```
# [1] "hello123" "456"      "a1b2"
```

```r
grep('^[0-9]+$', x, value = TRUE)    # all digits
```

```
# [1] "456"
```

Anchors, Quantifiers, Alternation

```r
x <- c('cat', 'concatenate', 'scat', 'catalog')
grep('^cat', x, value = TRUE)     # starts with 'cat'
```

```
# [1] "cat"     "catalog"
```

```r
grep('cat$', x, value = TRUE)     # ends with 'cat'
```

```
# [1] "cat"  "scat"
```

```r
x <- c('cat', 'dog', 'catfish', 'dogfish', 'bird')
grep('cat|dog', x, value = TRUE)       # cat OR dog
```

```
# [1] "cat"     "dog"     "catfish" "dogfish"
```

```r
grep('^(cat|dog)$', x, value = TRUE)   # exactly cat or dog
```

```
# [1] "cat" "dog"
```

Capture Groups and Backreferences

```r
# swap first and last name
x <- c('John Smith', 'Jane Doe', 'Bob Lee')
sub('(\\w+) (\\w+)', '\\2, \\1', x)
```

```
# [1] "Smith, John" "Doe, Jane"   "Lee, Bob"
```

```r
# extract parts of a date
dates <- c('2024-01-15', '2023-12-31')
sub('(\\d{4})-(\\d{2})-(\\d{2})', 'Year:\\1, Month:\\2', dates)
```

```
# [1] "Year:2024, Month:01" "Year:2023, Month:12"
```

Greedy vs Lazy Matching

```r
x <- '<b>bold</b> and <i>italic</i>'
regmatches(x, gregexpr('<.*>', x))     # greedy
```

```
# [[1]]
# [1] "<b>bold</b> and <i>italic</i>"
```

```r
regmatches(x, gregexpr('<.*?>', x))    # lazy (non-greedy)
```

```
# [[1]]
# [1] "<b>"  "</b>" "<i>"  "</i>"
```

Lookahead and Lookbehind (Perl)

```r
x <- c('$100', '200', '$300', '400')
regmatches(x, gregexpr('(?<=\\$)\\d+', x, perl = TRUE))
```

```
# [[1]]
# [1] "100"
#
# [[2]]
# character(0)
#
# [[3]]
# [1] "300"
#
# [[4]]
# character(0)
```

```r
# negative lookahead: files NOT ending in .txt
x <- c('file.txt', 'image.png', 'doc.txt', 'photo.jpg')
grep('\\.(?!txt)', x, value = TRUE, perl = TRUE)
```

```
# [1] "image.png" "photo.jpg"
```

regexec() — Full Capture Group Extraction

```r
x <- c('2024-01-15 14:30', '2023-12-31 09:00')
pattern <- '(\\d{4})-(\\d{2})-(\\d{2}) (\\d{2}):(\\d{2})'
m <- regexec(pattern, x)
regmatches(x, m)
```

```
# [[1]]
# [1] "2024-01-15 14:30" "2024"             "01"               "15"
# [5] "14"               "30"
#
# [[2]]
# [1] "2023-12-31 09:00" "2023"             "12"               "31"
# [5] "09"               "00"
```

Practical Regex Examples

```r
# validate email (simplified)
emails <- c('user@example.com', 'bad@@email',
            'test@domain.org', 'no_at_sign')
grep('^[\\w.]+@[\\w.]+\\.[a-z]{2,}$', emails,
     value = TRUE, perl = TRUE)
```

```
# [1] "user@example.com" "test@domain.org"
```

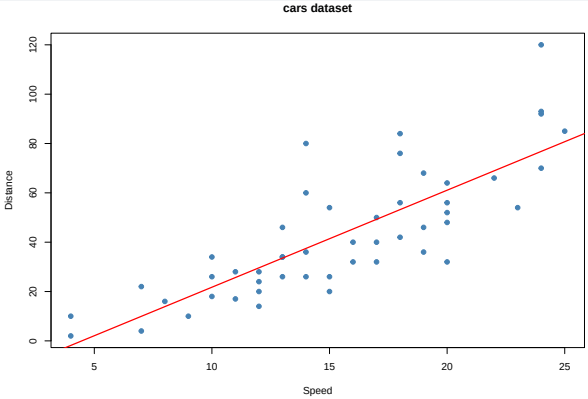
```r
# clean column names: replace non-alphanumeric with _
col_names <- c('First Name', 'Last.Name', 'Age (years)')
gsub('[^a-zA-Z0-9]', '_', col_names)
```

```
# [1] "First_Name"  "Last_Name"   "Age__years_"
```

Graphics

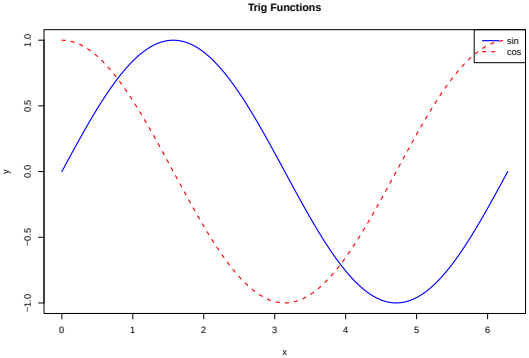
Scatter Plots

```r
plot(cars$speed, cars$dist, xlab = 'Speed', ylab = 'Distance',
     main = 'cars dataset', pch = 19, col = 'steelblue')
abline(lm(dist ~ speed, data = cars), col = 'red', lwd = 2)
```



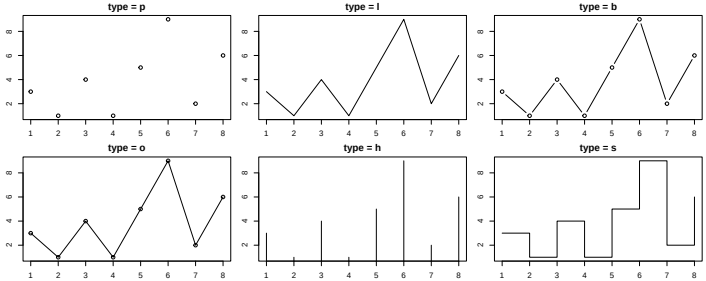
Essential Plot Parameters

```r
x <- seq(0, 2*pi, length.out = 100)
plot(x, sin(x), type = 'l', col = 'blue', lwd = 2,
     main = 'Trig Functions', xlab = 'x', ylab = 'y')
lines(x, cos(x), col = 'red', lwd = 2, lty = 2)
legend('topright', legend = c('sin', 'cos'),
       col = c('blue', 'red'), lty = c(1, 2), lwd = 2)
```



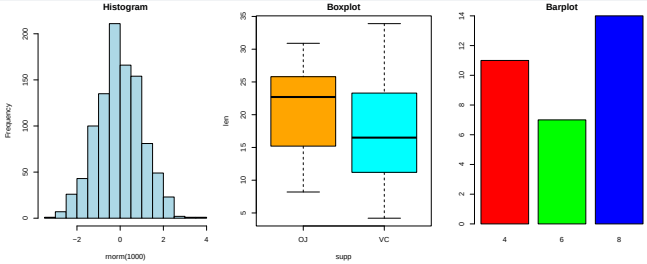
Plot Types: type Argument

```
par(mfrow = c(2, 3), mar = c(2,2,2,1))
y <- c(3,1,4,1,5,9,2,6)
for (tp in c('p','l','b','o','h','s')) {
  plot(y, type = tp, main = paste("type =", tp))
}
```



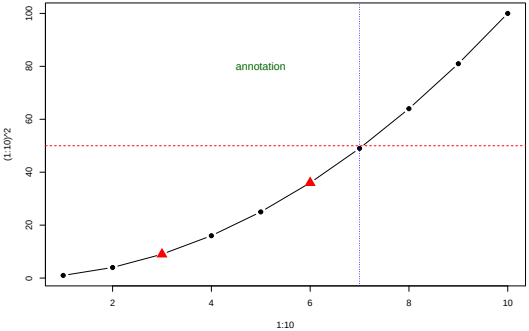
Multi-panel Plots with par(mfrow)

```
par(mfrow = c(1, 3), mar = c(4,4,2,1))
hist(rnorm(1000), main = 'Histogram', col = 'lightblue')
boxplot(len ~ supp, data = tg, main = 'Boxplot',
        col = c('orange', 'cyan'))
barplot(table(mtcars$cyl), main = 'Barplot',
        col = c('red', 'green', 'blue'))
```



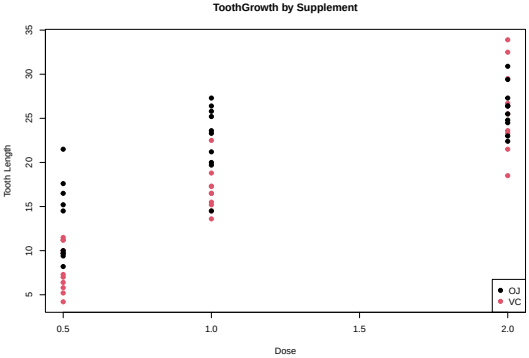
Adding Elements to Plots

```r
plot(1:10, (1:10)^2, type = 'b', pch = 19)
abline(h = 50, col = 'red', lty = 2)
abline(v = 7, col = 'blue', lty = 3)
text(5, 80, 'annotation', col = 'darkgreen', cex = 1.2)
points(c(3, 6), c(9, 36), pch = 17, col = 'red', cex = 2)
```



Color by Factor and Saving Plots

```
plot(tg$dose, tg$len, col = as.factor(tg$supp),
     pch = 19, xlab = 'Dose', ylab = 'Tooth Length',
     main = 'ToothGrowth by Supplement')
legend('bottomright', legend = levels(as.factor(tg$supp)),
       col = 1:2, pch = 19)
```




```
png('my_plot.png', width = 800, height = 600)
plot(1:10, main = 'My Plot'); dev.off()
pdf('my_plot.pdf', width = 8, height = 6)
```

Date and Time

Date Objects

```r
today <- Sys.Date(); today; class(today)
```

```
# [1] "2026-03-23"
# [1] "Date"
```

```r
as.Date('2024-01-15')
```

```
# [1] "2024-01-15"
```

```r
as.Date('15/01/2024', format = '%d/%m/%Y')
```

```
# [1] "2024-01-15"
```

```r
as.Date('Jan 15, 2024', format = '%b %d, %Y')
```

```
# [1] NA
```

```r
today + 30
```

```
# [1] "2026-04-22"
```

```r
as.Date('2024-12-31') - as.Date('2024-01-01')
```

```
# Time difference of 365 days
```

Date Formatting

```r
d <- as.Date('2024-07-15')
format(d, '%Y'); format(d, '%B'); format(d, '%A')
```

```
# [1] "2024"
# [1] " "
# [1] " "
```

```r
format(d, '%d %b %Y')
```

```
# [1] "15  7  2024"
```

```r
weekdays(d); months(d); quarters(d)
```

```
# [1] " "
# [1] " "
# [1] "Q3"
```

Date-Time: POSIXct and POSIXlt

```r
now <- Sys.time(); now; class(now)
```

```
# [1] "2026-03-23 08:38:09 CST"
# [1] "POSIXct" "POSIXt"
```

```r
as.POSIXct('2024-01-15 14:30:00', tz = 'Asia/Taipei')
```

```
# [1] "2024-01-15 14:30:00 CST"
```

```r
# POSIXlt: list-based access to components
lt <- as.POSIXlt(now)
lt$hour; lt$min; lt$wday    # 0=Sunday
```

```
# [1] 8
# [1] 38
# [1] 1
```

difftime, Sequences, and Format Codes

```r
d1 <- as.Date('2024-01-01'); d2 <- as.Date('2024-12-31')
difftime(d2, d1, units = 'weeks')
```

```
# Time difference of 52.14286 weeks
```

```r
seq(as.Date('2024-01-01'), as.Date('2024-06-01'), by = 'month')
```

```
# [1] "2024-01-01" "2024-02-01" "2024-03-01" "2024-04-01" "2024-05-01"
# [6] "2024-06-01"
```

Code	Meaning	Code	Meaning
`%Y`	4-digit year	`%H`	hour (00–23)
`%m`	month (01–12)	`%M`	minute (00–59)
`%d`	day (01–31)	`%S`	second (00–59)
`%B`/`%b`	month name/abbrev	`%A`/`%a`	weekday name/abbrev

strftime() and strptime()

```r
strftime(Sys.time(), '%Y-%m-%d %H:%M:%S %Z')
```

```
# [1] "2026-03-23 08:38:09 CST"
```

```r
strptime('15-Jan-2024 14:30', format = '%d-%b-%Y %H:%M')
```

```
# [1] NA
```

File I/O

Reading and Writing Text Files

```r
df <- read.csv('data.csv')
df <- read.csv('data.csv', header = TRUE, stringsAsFactors = FALSE)
df <- read.delim('data.tsv')                # tab-separated
df <- read.table('data.txt', header = TRUE, sep = '|')
```

```r
write.csv(df, 'output.csv', row.names = FALSE)
write.table(df, 'output.tsv', sep = '\t',
            row.names = FALSE, quote = FALSE)
```

Reading and Writing R Objects

```r
# save multiple R objects (binary, preserves types)
save(x, y, file = 'my_data.RData')
load('my_data.RData')

# save/load a single object
saveRDS(df, 'my_df.rds')
df <- readRDS('my_df.rds')
```

readLines, writeLines, scan

```r
lines <- readLines('file.txt')
lines <- readLines('file.txt', n = 10)     # first 10 lines
writeLines(c('line 1', 'line 2'), 'out.txt')
nums  <- scan('numbers.txt')               # numeric
words <- scan('words.txt', what = '')       # character
```

File System Functions

```
getwd(); setwd('/path/to/dir')
list.files(); list.files(pattern = '\\.csv$')
file.exists('myfile.csv'); dir.exists('mydir')
file.info('myfile.csv')
file.copy('src.csv', 'dst.csv')
file.rename('old.csv', 'new.csv')
file.remove('temp.csv')
dir.create('new_folder')
normalizePath('relative/path')       # full absolute path
```

Binary I/O and Downloads

```r
# practical: inspect raw type
x <- charToRaw("ABC"); x; typeof(x)
```

```
# [1] 41 42 43
# [1] "raw"
```

```r
rawToChar(x); as.integer(x)     # see the byte values
```

```
# [1] "ABC"
# [1] 65 66 67
```

```r
# write/read raw bytes
raw_data <- charToRaw("Hello, world!")
writeBin(raw_data, 'hello.bin')
dat <- readBin('hello.bin', what = 'raw', n = 100)
rawToChar(dat)
```

```r
# download files from the internet
download.file("https://example.com/data.csv", "data.csv")
unzip("archive.zip", exdir = "output_dir")
```

readline() and interactive()

```r
name <- readline(prompt = "Enter your name: ")
cat("Hello,", name, "\n")
n <- as.numeric(readline("Enter a number: "))
```

```r
if (interactive()) {
  cat("Running interactively\n")
} else {
  cat("Running in batch mode\n")
}
```

Error Handling

tryCatch() — Structured Error Handling

```r
# note: tryCatch catches the FIRST condition raised
result <- tryCatch({
  log('a')      # error: non-numeric argument
}, warning = function(w) {
  cat("Warning:", conditionMessage(w), "\n"); NA
}, error = function(e) {
  cat("Error:", conditionMessage(e), "\n"); NA
}, finally = {
  cat("(cleanup code runs always)\n")
})
```

```
# Error: non-numeric argument to mathematical function
# (cleanup code runs always)
```

```r
result
```

```
# [1] NA
```

`try()` — Simple Error Handling

```r
result <- try(log('a'), silent = TRUE)
inherits(result, 'try-error')
```

```
# [1] TRUE
```

```r
# useful in loops: skip failing inputs
inputs <- list(1, 'a', 4, NULL, 9)
results <- list()
for (i in seq_along(inputs)) {
  results[[i]] <- try(sqrt(inputs[[i]]), silent = TRUE)
}
Filter(\(r) !inherits(r, 'try-error'), results)
```

```
# [[1]]
# [1] 1
#
# [[2]]
# [1] 2
#
# [[3]]
# [1] 3
```

Common Error-Handling Idioms

```r
# silently return NULL on failure
safe_parse <- function(x) {
  tryCatch(as.numeric(x), warning = function(w) NULL,
                          error   = function(e) NULL)
}
safe_parse("3.14"); safe_parse("abc"); safe_parse(NULL)
```

```
# [1] 3.14
# NULL
# numeric(0)
```

```r
# wrap risky operations with a default value
safe_log <- function(x, default = NA) {
  tryCatch(log(x), error = function(e) default)
}
sapply(list(10, -1, 'a', NULL), safe_log)
```

```
# [1] 2.302585      NaN       NA       NA
```

stop(), warning(), message()

```r
safe_div <- function(a, b) {
  if (b == 0) stop("Division by zero!")
  a / b
}
safe_div(10, 2); safe_div(10, 0)
```

```
# [1] 5
# Error in safe_div(10, 0): Division by zero!
```

```r
check_val <- function(x) {
  if (x < 0) warning("Negative value!")
  sqrt(abs(x))
}
check_val(-4)
```

```
# [1] 2
```

stopifnot() — Assertions

```r
my_mean <- function(x) {
  stopifnot(is.numeric(x), length(x) > 0)
  sum(x) / length(x)
}
my_mean(1:5)
```

```
# [1] 3
```

```r
my_mean('hello')
```

```
# Error in my_mean("hello"): is.numeric(x) is not TRUE
```

Useful Utilities

do.call()

```r
dfs <- list(
  data.frame(x = 1:2, y = 3:4),
  data.frame(x = 5:6, y = 7:8),
  data.frame(x = 9:10, y = 11:12))
do.call(rbind, dfs)
```

```
#    x  y
# 1  1  3
# 2  2  4
# 3  5  7
# 4  6  8
# 5  9 11
# 6 10 12
```

```r
args <- list(1:10, na.rm = TRUE)
do.call(mean, args)
```

```
# [1] 5.5
```

`Vectorize()` — Scalar to Vector

```r
f <- function(x, y) {
  if (x > y) x - y else x + y
}
f_vec <- Vectorize(f)
f_vec(1:3, 2)
```

```
# [1] 3 4 1
```

table() and prop.table()

```r
x <- c('A','B','A','C','B','A','C','C','B','A')
table(x); prop.table(table(x))
```

```
# x
# A B C
# 4 3 3
```

```
# x
#   A   B   C
# 0.4 0.3 0.3
```

```r
y <- c(1,1,2,1,2,2,1,2,1,2)
addmargins(table(x, y))
```

```
#      y
# x      1  2 Sum
#   A    1  3   4
#   B    2  1   3
#   C    2  1   3
#   Sum  5  5  10
```

match.arg() and missing()

```r
my_plot <- function(type = c('scatter', 'line', 'bar')) {
  type <- match.arg(type)
  cat("Plotting:", type, "\n")
}
my_plot()         # default: first option
```

```
# Plotting: scatter
```

```r
my_plot('l')      # partial matching
```

```
# Plotting: line
```

```r
f <- function(x, y) {
  cat("nargs:", nargs(), "\n")
  if (missing(y)) cat("y is missing\n")
}
f(1); f(1, 2)
```

```
# nargs: 1
# y is missing
# nargs: 2
```

invisible() — Silent Return

```r
# invisible: returns value without printing
f <- function(x) invisible(x^2)
f(5)                  # nothing printed
y_val <- f(5); y_val # but value is captured
```

```
# [1] 25
```

```r
# common pattern: side-effect functions return invisible(NULL)
my_print <- function(x) {
  cat("Value:", x, "\n")
  invisible(NULL)
}
result <- my_print(42); is.null(result)
```

```
# Value: 42
```

```
# [1] TRUE
```

on.exit() — Guaranteed Cleanup

```r
safe_read <- function(path) {
  con <- file(path, 'r')
  on.exit(close(con))       # guaranteed cleanup
  readLines(con)
}
```

```r
# add = TRUE: stack multiple on.exit calls
process <- function(path) {
  tmp <- tempfile()
  on.exit(unlink(tmp), add = TRUE)      # cleanup 1
  con <- file(path, 'r')
  on.exit(close(con), add = TRUE)       # cleanup 2
}
```

Recursive Functions and `Recall()`

```r
my_factorial <- function(n) {
  if (n <= 1) return(1)
  n * Recall(n - 1)
}
my_factorial(5)
```

```
# [1] 120
```

```r
fib <- function(n) {
  if (n <= 2) return(1)
  Recall(n - 1) + Recall(n - 2)
}
sapply(1:10, fib)
```

```
#  [1]  1  1  2  3  5  8 13 21 34 55
```

Local Helper Functions

```r
search_with_retry <- function(query, max_tries = 3) {
  is_valid <- function(result) {
    !is.null(result) && length(result) > 0
  }
  for (i in seq_len(max_tries)) {
    result <- tryCatch(grep(query, letters, value = TRUE),
                       error = function(e) NULL)
    if (is_valid(result)) return(result)
    Sys.sleep(0.1)
  }
  NULL
}
search_with_retry("^[a-c]")
```

```
# [1] "a" "b" "c"
```

Naming Conventions and Dot-Prefix

```r
# names starting with . are "hidden" (not shown by ls())
e <- new.env()
e$.internal <- function(x) x + 1
e$public    <- function(x) x * 2
ls(e)                          # .internal not shown
```

```
# [1] "public"
```

```r
ls(e, all.names = TRUE)       # now it's shown
```

```
# [1] ".internal" "public"
```

- Convention: `.name` signals "internal/private"
- Many base R internals use this: `.Machine`, `.Platform`, `.GlobalEnv`

Sys.sleep(), Sys.info(), Sys.time()

```r
Sys.sleep(1.5)              # pause execution for 1.5 seconds
Sys.time()                  # current date-time (POSIXct)
Sys.info()                  # OS name, machine, user, etc.
.Platform$OS.type           # "unix" or "windows"
.Machine$double.eps         # machine epsilon
```

Benchmarking: `system.time()` and `proc.time()`

```r
system.time({
  x <- matrix(rnorm(1e6), 1000, 1000)
  y_mat <- crossprod(x)
})
```

```
#    user  system elapsed
#   0.114   0.059   0.039
```

```r
pt <- proc.time()
x <- sum(1:1e7)
proc.time() - pt
```

```
#    user  system elapsed
#   0.005   0.013   0.001
```

system2() — Run External Commands

```r
system2("echo", args = c("hello", "world"))
out <- system2("date", stdout = TRUE); cat(out, "\n")
system2("sleep", args = "5", wait = FALSE)     # background
```

```r
# cross-platform branching
if (.Platform$OS.type == "windows") {
  system2("taskkill", args = c("/F", "/IM", "myapp.exe"),
          stdout = NULL, stderr = NULL)
} else {
  system2("pkill", args = "myapp",
          stdout = NULL, stderr = NULL)
}
```

Encoding() and iconv()

```
Encoding(x)
iconv(x, from = 'UTF-8', to = 'BIG5')
iconv(x, from = 'BIG5', to = 'UTF-8')
Sys.setlocale('LC_ALL', 'zh_TW.UTF-8')
```

Advanced Topics

S3 Object System Basics

```r
person <- list(name = 'Alice', age = 30)
class(person) <- 'Person'
print.Person <- function(x, ...) {
  cat(sprintf("Person: %s (age %d)\n", x$name, x$age))
}
person
```

```
# Person: Alice (age 30)
```

Formula Objects

```r
f <- y ~ x1 + x2
class(f); typeof(f); all.vars(f)
```

```
# [1] "formula"
# [1] "language"
# [1] "y"  "x1" "x2"
```

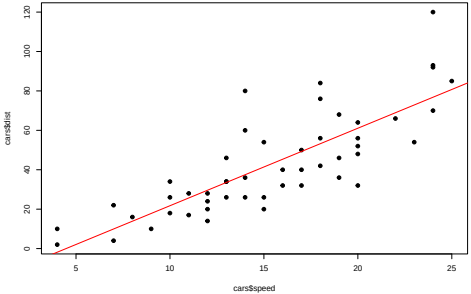
```r
y ~ x                  # simple regression
y ~ x1 + x2            # multiple regression
y ~ x1 * x2            # with interaction (= x1 + x2 + x1:x2)
y ~ .                   # all other columns
y ~ poly(x, 3)         # polynomial
y ~ log(x)             # transformed
```

Linear Regression Example

```r
model <- lm(dist ~ speed, data = cars)
model$coefficients
```

```
# (Intercept)       speed
#  -17.579095    3.932409
```

```r
plot(cars$speed, cars$dist, pch = 19)
abline(model, col = 'red', lwd = 2)
```



Random Number Generation

```r
set.seed(42)
runif(5); rnorm(5, mean = 0, sd = 1)
```

```
# [1] 0.9148060 0.9370754 0.2861395 0.8304476 0.6417455
# [1]  0.04788474 -1.10459944  0.53902380  0.58020632 -0.65750284
```

```r
rbinom(10, size = 1, prob = 0.5)
```

```
#  [1] 1 1 0 0 1 1 0 1 1 0
```

```r
rpois(10, lambda = 3)
```

```
#  [1] 3 2 5 3 5 4 4 2 4 0
```

```r
rexp(5, rate = 1)
```

```
# [1] 0.6658322 4.9959685 0.2235573 1.2113841 0.7190924
```

options() and getOption()

```r
options(digits = 15)            # more decimal places
options(scipen = 999)           # avoid scientific notation
getOption('digits')
options(warn = 2)               # warnings as errors
```

Package Management

```r
install.packages('dplyr')
library(dplyr)
require(dplyr)                    # returns TRUE/FALSE
requireNamespace('dplyr', quietly = TRUE)  # no attach
installed.packages()[, 'Package']
update.packages()
remove.packages('dplyr')
```

Help System and Useful One-Liners

```r
?prod                                # help on a function
example(cumsum)                      # run examples
help.search('multivariate normal')
help(mgcv::rmvn)                    # package::function help
```

```r
rm(list = ls())                     # clear workspace
suppressWarnings(as.integer(c('1', 'a', '3')))
sort(table(x), decreasing = TRUE)   # frequency count
source('my_script.R')
args <- commandArgs(trailingOnly = TRUE)
```